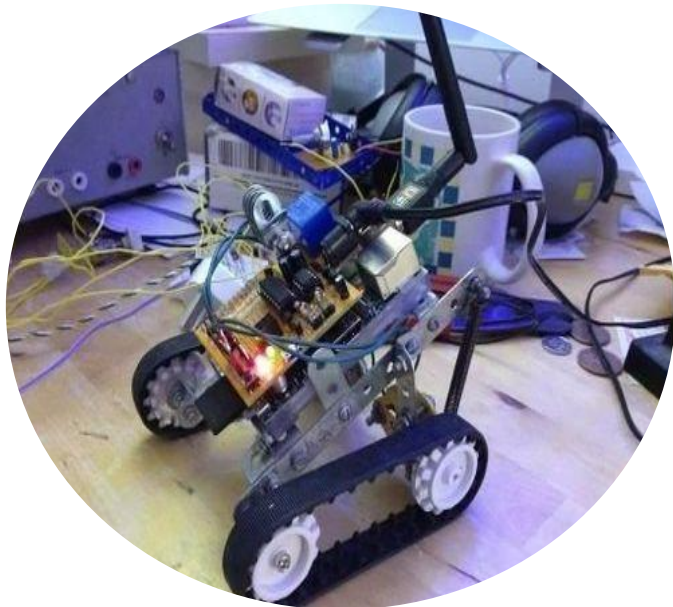
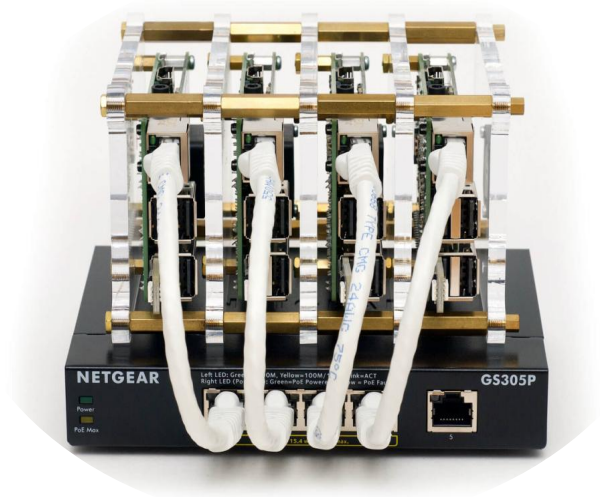




微机原理与微系统实验-树莓派

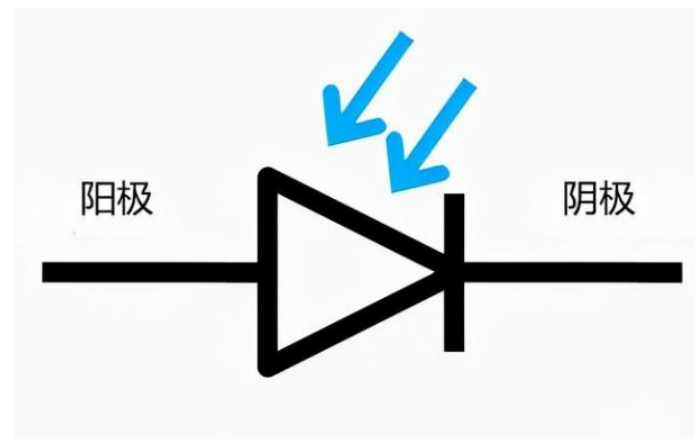
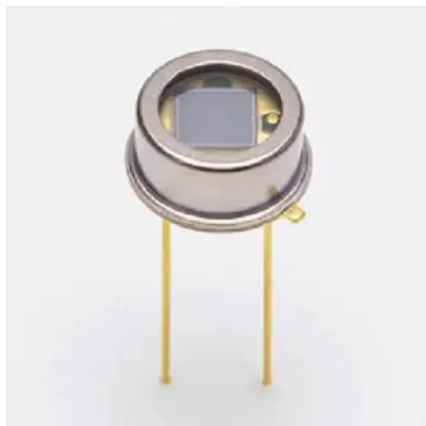


主讲老师：王小静
办公地点：慧园2栋411
open office hours：周三14:00-16:00

Class 10: 红外遥控器实验

光电二极管

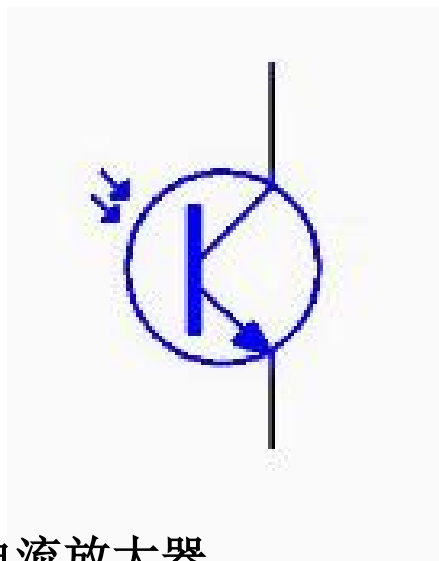
光敏二极管，又叫光电二极管（英语：**photodiode**）是一种将光信号转换为电信号的半导体器件，其核心部分是一个具有光敏特性的**PN结**，其对光的变化非常敏感，具有单向导电性，而且光强不同的时候会改变其电学特性，因此，可以利用光照强弱来改变电路中的电流。



光控电流源

光电三极管

光电三极管，也叫光敏三极管，基本结构和普通三极管一样，包含两个PN结。当光入射到基极表面，产生光生电子-空穴对，这些载流子在**b-c**结电场作用下分离，电子向集电极漂移，而空穴移向基极，致使基极电位升高。在**c、e**间外加电压作用下，大量电子由发射极注入，除少数在基极与空穴复合外，大量通过极薄的基极被集电极收集，成为输出光电流。光电三极管工作原理分为两个过程：一是光电转换；二是光电流放大。光电三极管可以看作是一个光电二极管与一个三极管的结合体，具有放大作用。



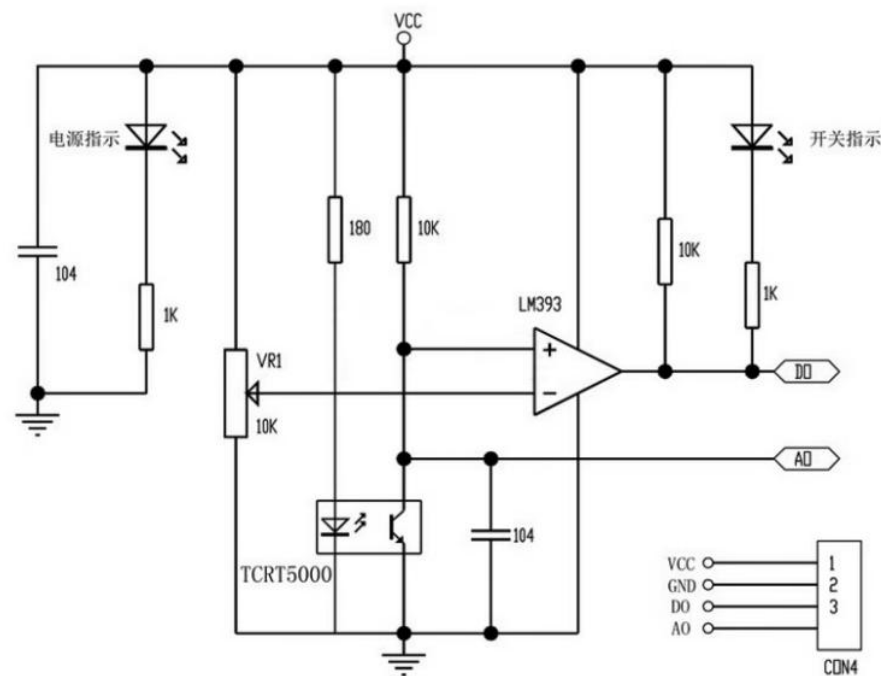
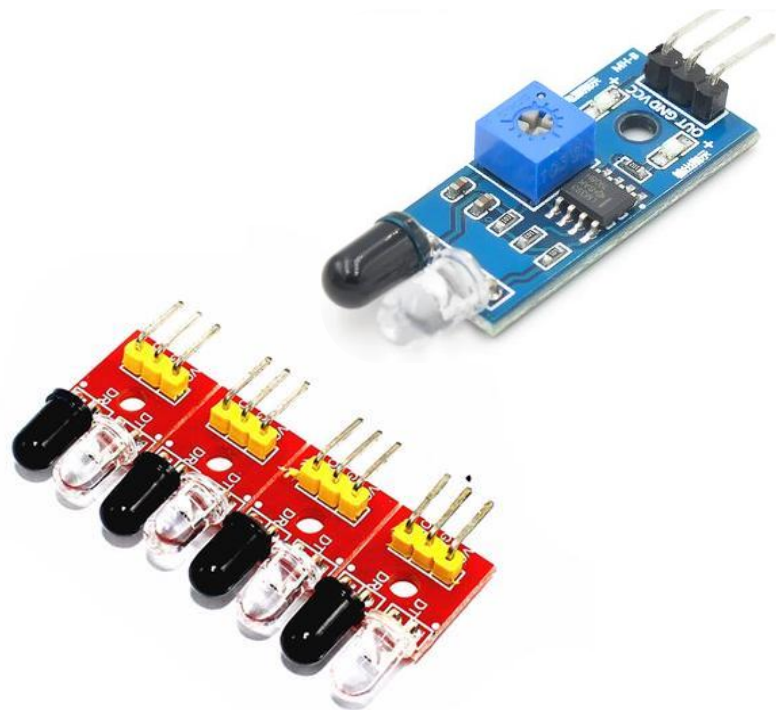
光控电流放大器

光电开关

光电开关的工作原理主要基于光电效应。光电开关通常由发射器、接收器和检测电路三部分组成。发射器（如发光二极管或激光二极管）发出光束，这些光束会射向目标物体。根据目标物体的特性（如反射、吸收或透射），部分或全部光束会被反射回来，并由光电开关的接收器接收。

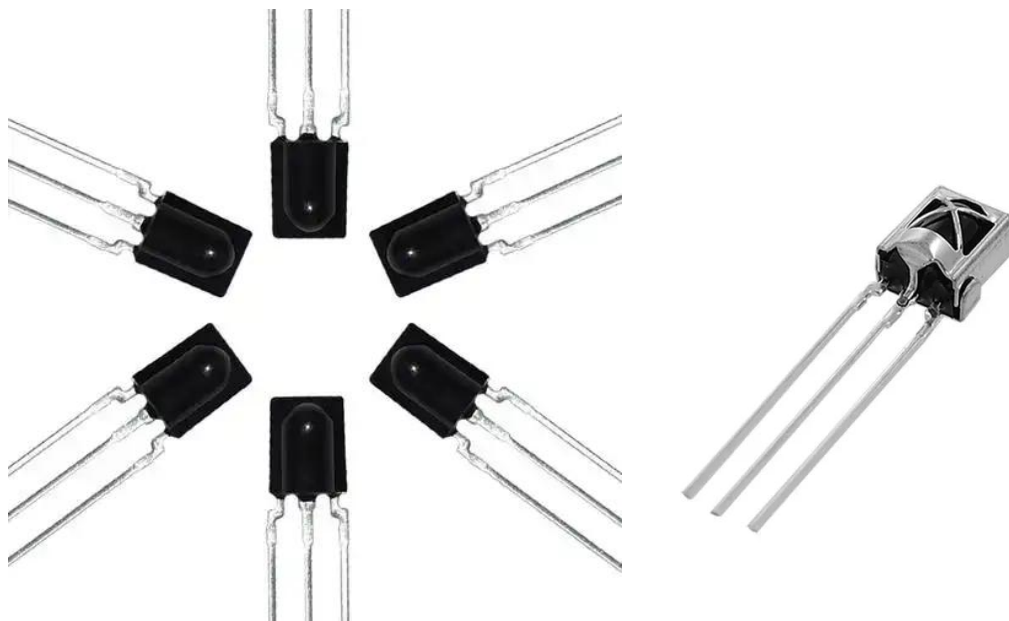


四路灰度循迹巡线模块



红外接收头/一体化接收头

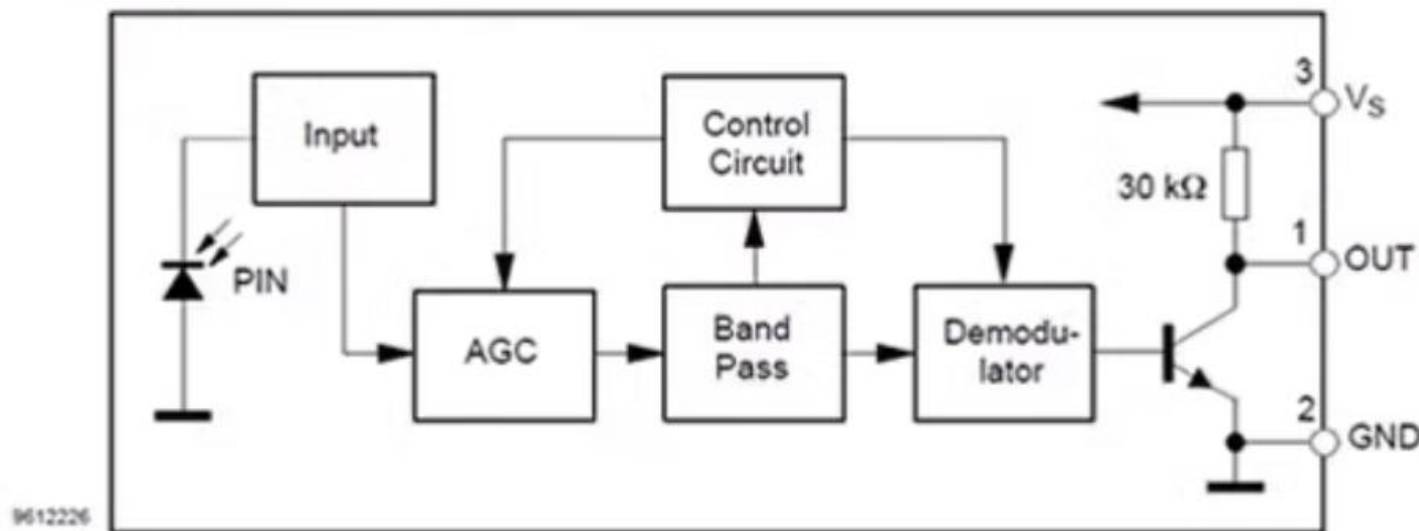
红外接收头是一个集成了光电二极管、放大器、解调电路等功能的完整模块，其内部结构与远超一个简单的光敏器件。



一体化接收头内部通常包含：

- ◆ 红外光电二极管：负责接收红外信号。
- ◆ 前置放大器：将光电二极管产生的微弱电流信号进行放大。
- ◆ 带通滤波器：中心频率通常为38kHz（也有36kHz, 40kHz等）。它只允许这个频率附近的信号通过。
- ◆ 解调器：从38kHz的载波信号中提取出原来的数字信号（即遥控器按下的指令编码）。
- ◆ 整形输出电路：将解调后的信号整形成干净的数字方波输出。

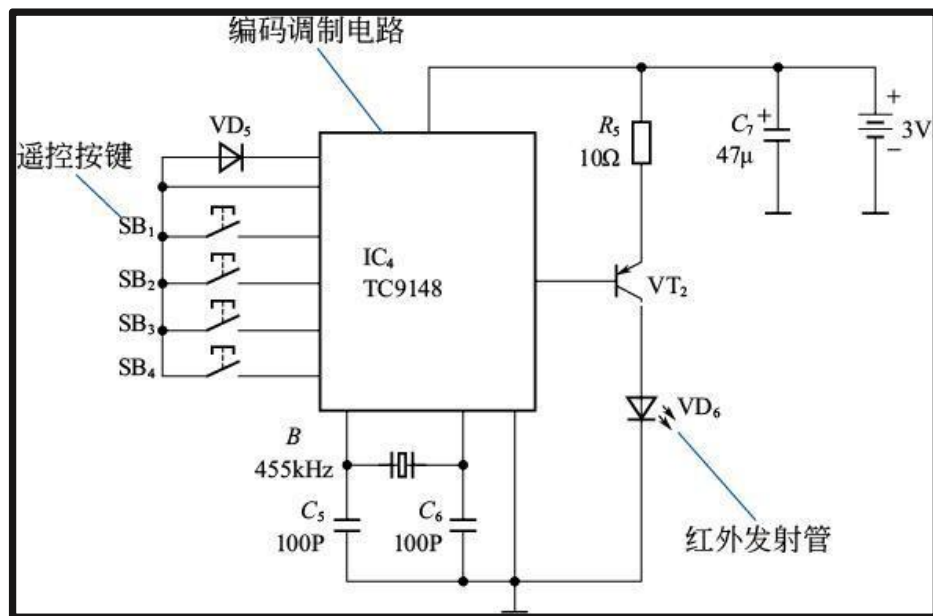
Block Diagram



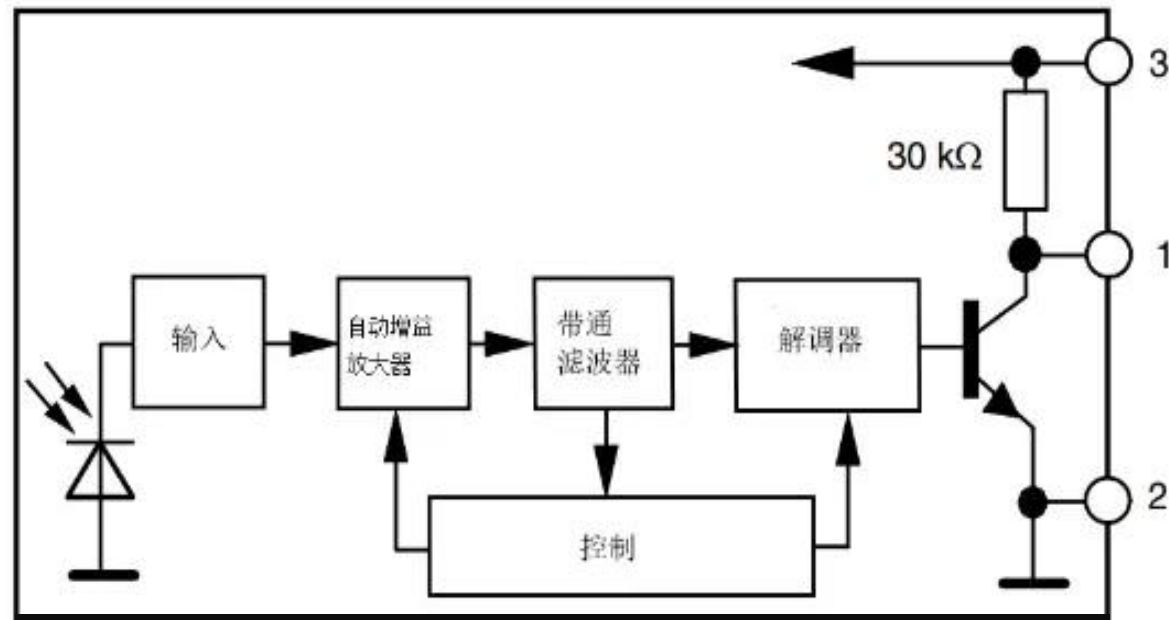
红外接收头通常与红外发射管配合使用，应用于遥控器、光电开关、光电记数设备等。

工作流程:

1. 遥控器发送信号：红外遥控器发出的信号并不是一个简单的“开”或“关”的光脉冲，而是一串复杂的、包含地址和命令信息的数字编码，并通过载波（例如**38KHz**）进行调制后发射出去的。
2. 一体化接收头的工作就是：接收微弱红光 -> 放大 -> 滤除非**38kHz**的干扰（如环境光、日光灯）-> 解调出数字编码 -> 输出干净的方波给单片机。



红外遥控发射器



红外接收器

编码：编码规定了数据是如何组织的，即“0”和“1”如何表示，以及一帧完整的数据包包含哪些信息。

红外遥控器的信号编码主要包括以下几种方法：

- ✓ 脉冲宽度编码（PWM）：这是家用电器中常用的编码方式。在这种编码方式中，宽脉冲代表二进制“1”，窄脉冲代表二进制“0”。例如，NEC编码协议就采用了这种编码方式¹²。
- ✓ 脉冲相位编码：在这种编码方式中，每个时间周期内都有一个上升沿或下降沿。如果检测到上升沿，则代表二进制“1”；如果检测到下降沿，则代表二进制“0”³。
- ✓ 脉冲间距编码（PPM）：在这种编码方式中，脉冲宽度相同，但脉冲之间的距离与所需编码的信息相关。例如，一个脉冲后面的低电平持续时间为 T ，则表示二进制“0”；如果低电平持续时间为 $2T$ ，则表示二进制“1”³。

红外遥控器的信号传输协议：

- ✓ **NEC传输协议**：这是目前红外遥控中使用最普遍的信号传输协议。NEC协议采用脉冲间距编码来表示信号。一帧数据包括引导码、地址码、数据码和数据反码。引导码由一个9ms的低电平和一个4.5ms的高电平组成，地址码和数据码均为8位，后面紧跟着8位反码，以确保指令的正确性²³。
- ✓ Philips RC-5协议：这种协议采用脉冲位置调制（PPM）码，通过脉冲串之间的时间间隔来实现信号调制¹。
- ✓ Sony SIRC编码：这是索尼公司使用的红外编码方式，具有较高的抗干扰能力⁴。
- ✓ Samsung标准：这是三星公司使用的红外编码方式，具有较高的传输速率和稳定性⁴。
- ✓ RAW码：这是一种原始的编码方式，不进行任何调制，直接传输二进制数据⁴。

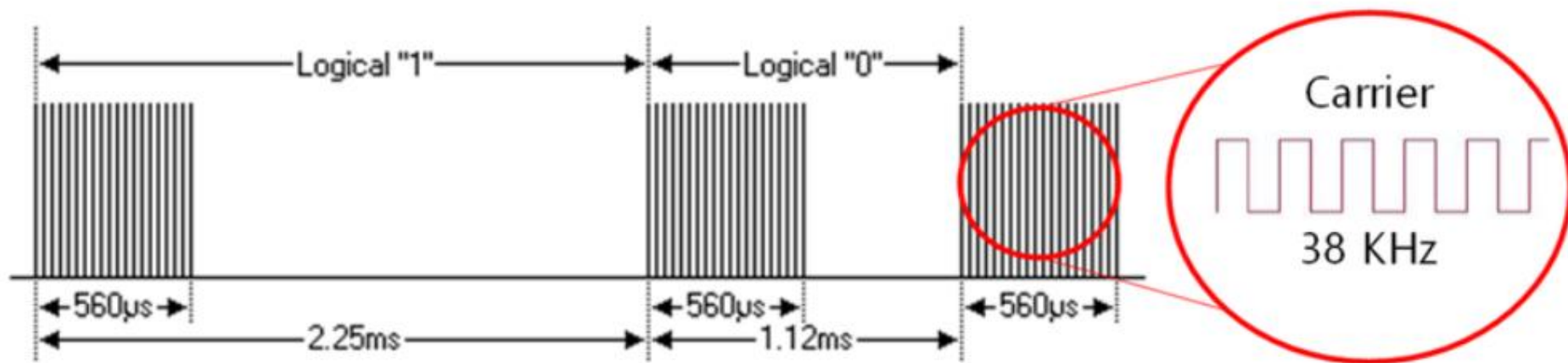
这些编码方式和协议的选择取决于具体的红外遥控器设计和应用需求。

[常见红外协议整理](#)

NEC编码说明:

1:用38khz载波的方式传播，如何区分逻辑0和1:

- 一个逻辑1传输需要2.25ms(560us脉冲+1680us低电平)
- 一个逻辑0的传输需要1.125ms(560us脉冲+560us低电平)。



而遥控接收头在收到脉冲的时候为低电平,在没有脉冲的时候为高电平,这样,我们在接收头端收到的信号为:逻辑1应该是560us低+1680us高,逻辑0应该是560us低+560us高。

2: NEC遥控器指令格式

NEC遥控指令的数据格式为:引导码、地址码、地址反码、控制码（按键值）、控制反码（按键值反码）、结束码。

- ◆ 引导码：由一个9ms的低电平和一个 4.5ms的高电平组成；
- ◆ 地址码、地址反码、控制码、控制反码：均是8位数据格式。按照低位在前,高位在后的顺序发送。采用反码是为了增加传输的可靠性(可用于校验)。
- ◆ 结束码：0.56ms的高电平



重复帧部分省略，道理相同

3：重复码

当用户长时间按住一个按键时，遥控器不会一直发送完整的数据帧（那样太耗电），而是先发送一次完整帧，然后每隔 110ms 发送一个特殊的重复码。

重复码结构：一个 9ms 脉冲 + 2.25ms 低电平 + 一个 560 μ s 脉冲。

接收头收到重复码后，会输出与上一次相同的解码结果。这实现了“长按”连续操作的功能。

遥控解码:



解码就只需要获取每个波形高低电平时长,就可以知道收到的数据是1还是0,最后读取整个数据即可。

过程：一体化接收头内部的光电二极管接收到光信号，内部的芯片会：

- 放大微弱信号。
- 带通滤波：只允许38kHz附近的信号通过，滤除所有其他频率的干扰光。
- 解调：检测到有38kHz载波时，输出低电平；检测不到38kHz载波时，输出高电平。注意这个反相关系：
发射端有载波（LED亮）-> 接收端输出低电平。发射端无载波（LED灭）-> 接收端输出高电平。
- 结果：一体化接收头最终输出的，就是一个与原始编码数据波形完全一致（但可能反向）的、干净的数字方波。这个方波可以直接送给单片机（MCU）的IO口进行解码。



对码：对码电视遥控器一般就是按住对应品牌按键不放，直到电视出现音量图标，然后马上松手即可完成对码。

总结：从按下按键到执行命令的全过程

1. 按下按键：单片机读取到哪个键被按下，查表找到对应的**NEC**编码（地址码+命令码）。
2. 调制与发射：单片机生成**NEC**编码波形，并用此波形去控制一个**38kHz**的**PWM**输出，驱动红外**LED**发射出已被调制的红外光。
3. 接收与解调：一体化接收头收到光信号，经过放大、滤波、解调后，还原出**NEC**编码波形，输出给设备（如电视）的主控**MCU**。
4. 解码与执行：设备的主控**MCU**检测到引导码，开始按**NEC**协议解析后续的**32**位数据。校验地址码是否正确，如果正确，则执行相应的命令码（如调节音量）。

分类	命令	含义
目录切换	pwd	# 显示当前所在目录
	cd ~	#切换到主目录（/home/pi）.~可以不写
	cd dir	#切换到指定目录，dir表示文件路径
	cd ..	#切换到上一级目录
文件展示	ls	#展示当前目录下所有的文件和文件夹（不包含隐藏文件）
	ls -a	#展示当前目录下所有的文件和文件夹（包含隐藏文件）
创建	touch file	#创建文件file
	mkdir dir	#创建目录dir
查看	cat file	#查看文件file内容
	more file	#查看文件file内容
	head file	#查看文件file前10行
	tail file	#查看文件file后10行
删除	rm file	#删除文件file
	rm -r dir	#删除目录dir
重命名或移动	mv file 1 file2	#若file2不存在，则将file1改名为file2； 若file2存在，则file1覆盖掉原file2，并更名为file2
	mv file dir	#将文件file移动到dir目录中

文本编辑器 (Text Editor)

- 在树莓派（或 Linux 系统）中，`nano` 和 `vi` 都是命令行下的文本编辑器，用于在终端中编辑文件（如代码、配置文件等）。两者的设计理念和方式使用差异较大。
 - `nano` 是一款轻量、简单、新手友好的文本编辑器，设计初衷是让用户能快速上手。它的特点是操作直观，底部会直接显示常用快捷键提示，不需要记忆复杂命令，适合初学者。
 - `vi` 是一款历史悠久、功能强大的文本编辑器，在所有 Linux 系统中几乎都预装。它的特点是模式化操作（需切换命令 / 插入模式），初期学习成本较高，但熟练后效率极高，适合编辑代码或配置文件。

文本编辑器	nano	vi	
基本用法	1. 打开 / 创建文件：nano 文件名（如 nano test.txt） 2. 编辑：直接输入文字（默认进入编辑模式） 3. 保存：Ctrl + O（回车确认文件名） 4. 退出：Ctrl + X	1. 打开 / 创建文件：vi 文件名（如 vi test.txt） 2. 进入编辑：命令模式下按 i（插入模式） 3. 保存退出：按 Esc 回命令模式，输入 :wq 并回车 4. 放弃退出：命令模式下输入 :q! 并回车	
常用快捷键	- Ctrl + O：保存文件 - Ctrl + X：退出编辑器 - Ctrl + K：剪切当前行 - Ctrl + U：粘贴剪切内容 - Ctrl + W：搜索文本 - Ctrl + \：替换文本 - Ctrl + C：显示光标位置	命令模式 - i：进入插入模式（在光标前编辑） - dd：删除当前行 - yy：复制当前行 - p：粘贴内容 - u：撤销操作 - Shift + zz：快速保存并退出 - h/j/k/l：左 / 下 / 上 / 右移动光标	末行模式 - :w：保存文件 - :q：退出（未保存时提示） - :q!：强制退出（不保存） - :wq：保存并退出 - :set nu：显示行号 - :set nonu：隐藏行号

树莓派解码红外接收头的信号

树莓派可以通过**lirc**库发射和接收红外信号；

1. 安装lirc库

```
1 | sudo apt-get update
2 | sudo apt-get install lirc
```

2. 修改配置文件

```
1 | sudo nano /boot/firmware/config.txt
```

修改 config.txt，添加以下两行代码：

Uncomment this to enable infrared communication.

dtoverlay=gpio-ir,gpio_pin=17 → 红外接收引脚

dtoverlay=gpio-ir-tx,gpio_pin=18 → 红外发送引脚

Ctrl+o 保存，Ctrl+x 退出

3. 修改驱动文件

```
1 | sudo nano /etc/lirc/lirc_options.conf
```

把：

driver = devinput

device = auto

修改为：

driver = default

device = /dev/lirc1

Ctrl+o 保存，Ctrl+x 退出

4. 重启树莓派

```
1 | sudo reboot
```

先把红外模块接好!!! **BCM17**引脚

5.测试

```
1 | sudo service lircd stop
2 | mode2 -d /dev/lirc1
```

停止lirc服务

测试模式：按下按键，终端会显示脉冲信号

此时拿着遥控器对着红外接收器随便按按键，就会打印出类似这样的信息

6.查看可用的遥控键值

屏幕输出：

```
KEY_0
KEY_1
KEY_102ND
KEY_10CHANNELSDOWN
KEY_10CHANNELSUP
KEY_2
KEY_3
KEY_3D_MODE
KEY_4
KEY_5
KEY_6
KEY_7
KEY_8
KEY_9
KEY_A
KEY_AB
KEY_ADDRESSBOOK
KEY_AGAIN
KEY_ALS_TOGGLE
KEY_ALTERASE
KEY_ANGLE
```

```
1 | irrecord -l
```

```
space 534
pulse 602
space 1640
pulse 600
space 1643
pulse 600
space 520
pulse 611
space 1644
pulse 599
space 535
pulse 599
space 1641
pulse 602
space 532
pulse 599
space 1641
pulse 603
space 532
pulse 602
space 520
pulse 616
space 1638
```



Ctrl+c 退出

7.开始录制

指定红外接收设备

录制文件保存的路径

```
1 | irrecord -d /dev/lirc1 ~/lircd.conf
```

屏幕出现

```
1 | Press RETURN to continue.
```

按一下回车。等待屏幕出现

！ 注意在此期间不要按任何按键

```
1 | Enter name of remote (only ascii, no spaces) :
```

输入文件名，文件名只能纯英文字符并且不能有空格。 比如，键入： myir

回车后 屏幕出现

```
1 | Press RETURN now to start recording.
```

再次按下回车后，树莓派便开始记录按键。

！ 注意，如果回车后在**10s**内没有按下任何按键，命令就终止了。

这时需要轮流随机按遥控器上的按键，每按下一个按键（不是长按）会出现一个点。这里需要一些耐心。一直重复随机按下遥控器上的按键，直到屏幕出现

```
1 | Please enter the name for the next button (press <ENTER> to finish recording)
```

这里需要为你遥控器上的按键配置名字，名字只能从之前查看 可用按键名词里面取 参看第6步，例如 我这里输入 KEY_1 然后 按下回车 屏幕出现

```
1 | Now hold down button "KEY_1".
```

按下用遥控器的数字 1 键，然后屏幕又出现

```
1 | Please enter the name for the next button (press <ENTER> to finish recording)
```

输入 KEY_2 ，回车，等待屏幕出现

```
1 | Now hold down button "KEY_2".
```

按下遥控器的数字 2 ， 以此重复

你可以记录完遥控器上的有所按键。按回车结束记录，此时屏幕出现 如下图

```
Checking for toggle bit mask.  
Please press an arbitrary button repeatedly as fast as possible.  
Make sure you keep pressing the SAME button and that you DON'T HOLD  
the button down!.  
If you can't see any dots appear, wait a bit between button presses.
```

意思是按下回车后 需要重复的按遥控器上的同一个按键。注意不是长按！直到屏幕出现

```
1 | Successfully written config file myir.lircd.conf
```

8.查看并修改录制的键值文件

```
1 | sudo nano myir.lircd.conf
```

```
gap          40949
repeat_gap    499037
toggle_bit_mask 0x0
frequency     38000

begin codes
    KEY_1      0x00FD00FF
    KEY_2      0x00FD807F
    KEY_3      0x00FD40BF
end codes

end remote
```

此处多余一列删掉

9.复制该配置文件到Lirc目录下

```
1 | sudo cp myir.lircd.conf /etc/lirc/lircd.conf.d
```

10.修改文件名：重命名devinput.lircd.conf文件

```
1 | cd /etc/lirc/lircd.conf.d
2 | sudo mv devinput.lircd.conf devinput.lircd.dist
```

11.测试录制后的按键输出：

```
1 | sudo service lircd restart      → 启动lirc服务
2 | sudo lircd --nodaemon --device /dev/lirc1 --driver default → 手动前台启动 LIRC 服务并指定关键参数
3 | sudo irw                        → 测试映射：按下遥控器按键，终端应显示对应键名
```

12. 关联python程序:

```
1 | cd /etc/lirc
2 | sudo mv irexec.lircrc lircrc
```

13. 配置lircrc文件

```
1 | sudo nano lircrc
```

```
begin
    prog = lab9.py
    button = KEY_0
    config = echo "KEY_0"
end
begin
    prog = lab9.py
    button = KEY_1
    config = echo "KEY_1"
end
```

你自己的函数名

要处理的按键的键名

配置传递到python函数中的数据

Ctrl+o 保存, Ctrl+x 退出

14. 编写python程序

```
1 | import lirc
2 |
3 | def pasreset(data): #解析按键
4 |     if data == 'echo "KEY_1"':
5 |         print("1 按下") #遥控器按下1:
6 |     elif data == 'echo "KEY_2"':
7 |         print("2 按下") #遥控器按下2:
8 |     elif data == 'echo "KEY_3"':
9 |         print("3 按下") #遥控器按下3:
10 |
11 | with lirc.LircdConnection(" Lab9.py ",) as conn:
12 |     while True:
13 |         string = conn.readline()
14 |         pasreset(string)
```

创建与lirc守护进程的连接

读取红外遥控器发送的数据

with 语法

在Python中，`with`语句被用于包裹执行代码块，以便于管理资源，比如文件操作、数据库连接等。使用`with`语句可以确保即使在代码块中发生异常时，资源也能被正确地关闭和清理。

with`语句的基本语法：

```
with expression as variable:  
    # 代码块  
    # 使用variable
```

- `expression`：是一个上下文管理器，它必须返回一个上下文管理对象。
- `as variable`：是可选的，如果你不需要从上下文管理器中获取一个值，可以省略`as variable`部分。
- 代码块：是你希望在资源被管理的情况下执行的代码。

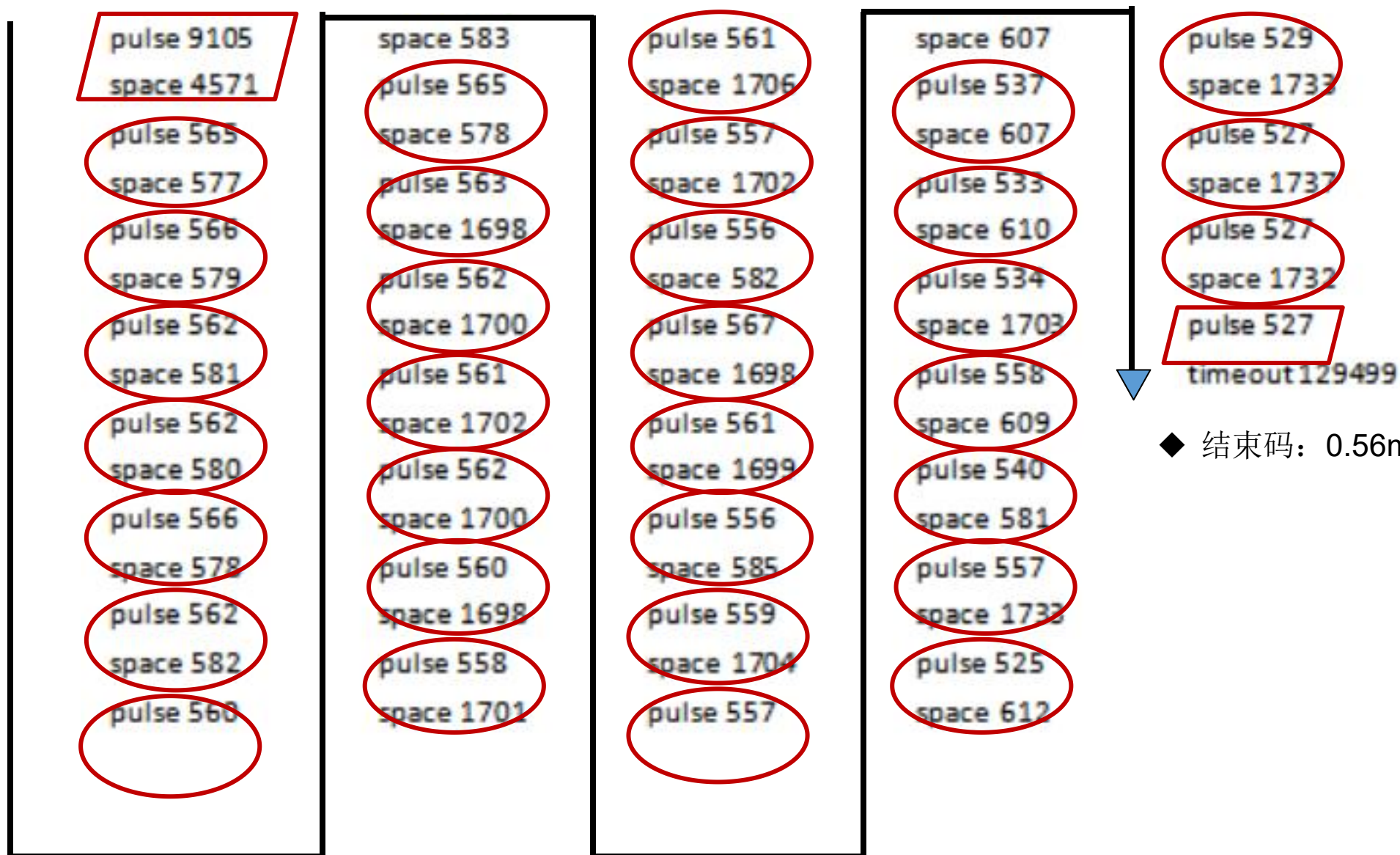
示例1：文件操作

```
with open('example.txt', 'r') as file:  
    content = file.read()  
    print(content)  
    # 文件在这里自动关闭`
```

在这个例子中，`open`函数返回一个文件对象，它是一个上下文管理器。`with`语句确保文件在代码块执行完毕后被关闭，即使在读取文件时发生异常。

`with`语句是Python中处理资源管理的一种优雅方式，它可以帮助减少代码中的样板代码，并提高代码的可读性和健壮性。

◆ 引导码：由一个9ms的低电平和一个 4.5ms的高电平组成；



◆ 结束码：0.56ms的高电平

实验要求

- 1: 按照指引配置，可以识别按键按下（至少录入三个按键值）；
- 2: 红外遥控控制红绿灯的三种状态： 红灯亮、绿灯亮、灭灯；

More(自愿):

遥控器变成钢琴按键，弹奏小星星。

